

JAVA BASICS

1. Data Types

A data type tells Java what kind of value a variable can store and what operations are valid for that value.

Primitive data types

byte	8-bit integer	byte age = 22;
short	16-bit integer	short marks = 450;
int	32-bit integer	int salary = 35000;
long	64-bit integer	long population = 1400000000L;
float	32-bit decimal	float price = 99.5f;
double	64-bit decimal	double pi = 3.14159;
char	single character	char grade = 'A';
boolean	true / false	boolean active = true;

Memory idea: primitive types store simple values directly; reference types store references to objects.

Real-time example: an employee record may use int for employeeId, double for salary, char for grade and boolean for active status.

2. Variables

A variable is a named memory location used to store a value that a program can read or change during execution.

Declaration and initialization

```
int age;  
age = 22;  
  
int salary = 35000;  
String name = "Ravi";
```

Declaration ☒ tells Java the type and variable name.

Initialization ☒ gives the variable its first value.

Types of Variables

Local variable	Declared inside a method or block and usable only within that scope.
Instance variable	Belongs to an object.
Static / class variable	Belongs to the class and is shared by all objects of that class.

Real-time: in a bank application, `accountNumber`, `balance` and `accountStatus` can be variables whose values change as transactions happen.

3. Operators

Operators are symbols that tell Java to perform an operation on one or more operands.

Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	calculations
Relational	<code>></code> <code><</code> <code>>=</code> <code><=</code> <code>==</code> <code>!=</code>	comparison & boolean
Logical	<code>&&</code> <code> </code> <code>!</code>	combine conditions
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code>	assign/update values
Unary	<code>++</code> <code>--</code> <code>+</code> <code>-</code> <code>!</code>	one operand
Ternary	<code>condition ? a : b</code>	short conditional choice
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>~</code>	bit-level operations
Shift	<code><<</code> <code>>></code> <code>>>></code>	shift bits

4. Arithmetic & Relational

```
int a = 10, b = 3;  
System.out.println(a + b); // 13  
System.out.println(a % b); // 1  
System.out.println(a > b); // true
```

Arithmetic operators are used for mathematical calculations. The % operator gives the remainder.

Relational operators

>, <, >=, <=, == and != compare values and produce a boolean result.

Important: == compares values for primitive types; = is assignment.

Real-time example

An e-commerce system can compare cartTotal >= minimumOrderValue before applying free delivery.

5. Logical, Assignment & Unary

```
int age = 22;
boolean citizen = true;
boolean eligible = age >= 18 && citizen;

int count = 10;
count += 5;      // 15
count++;         // 16
```

- ☒ && (AND) ☒ true only when both conditions are true.
- ☒ || (OR) ☒ true when at least one condition is true.
- ☒ ! (NOT) ☒ reverses a boolean value.
- ☒ Assignment operators store or update values.
- ☒ ++ / -- increase or decrease a value by 1.

Ternary operator

```
int marks = 72;
String result = marks >= 40 ? "Pass" : "Fail";
```

6. Decision-Making Statements

Decision-making statements allow a program to choose different execution paths based on conditions.

if	execute a block when a condition is true
if-else	choose between two paths
else-if ladder	test multiple conditions in order
nested if	place one decision inside another
switch	choose among multiple discrete cases

Use decisions when the program must answer a question such as: "Is the user eligible?", "Is payment successful?" or "Which menu option was selected?"

7. if, if-else & else-if

```
if (marks >= 40) {  
    System.out.println("Pass");  
}  
  
if (balance >= 500) {  
    withdraw();  
} else {  
    System.out.println("Insufficient balance");  
}
```

else-if ladder

```
if (marks >= 90) grade = 'A';  
else if (marks >= 75) grade = 'B';  
else if (marks >= 60) grade = 'C';  
else grade = 'D';
```

Conditions are checked from top to bottom. Once one condition is true, its block executes and the remaining ladder is skipped.

8. switch Statement

switch is useful when one expression is compared against several fixed case values.

```
int day = 2;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    default: System.out.println("Invalid");
}
```

- ❑ case ❑ a possible matching value.
- ❑ break ❑ exits the switch after a matched case.
- ❑ default ❑ runs when no case matches.

Real-time: a food-delivery app can use a menu option code to select Order Status, Cancel Order, Help, etc.

9. Loops

A loop repeatedly executes a block of code while a condition or iteration rule allows it.

for	best when the number of iterations is known or controlled by a counter
while	best when repetition depends mainly on a condition
do-while	runs the body at least once, then checks the condition
nested loop	a loop placed inside another loop

Real-time examples: print 1–100, process every item in a shopping cart, retry while a connection is unavailable, or process rows and columns in a matrix.

10. for, while & do-while

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

for ☐ initialization, condition and update are written together.

```
int i = 1;  
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

while ☐ checks the condition before each iteration. If it is false initially, the body may execute zero times.

```
int i = 1;  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

do-while ☐ executes the body first, so it executes at least once.

11. break, continue & Nested Loops

break

Immediately terminates the nearest loop or switch.

```
for (int i=1; i<=10; i++) {  
    if (i == 5) break;  
}
```

continue

Skips the remaining statements in the current iteration and moves to the next iteration.

Nested loop

A loop inside another loop. Commonly used for tables, matrices and row/column processing.

```
for (int row=1; row<=3; row++) {  
    for (int col=1; col<=3; col++) {  
        System.out.print("*");  
    }  
}
```

12. Quick Revision

Data type	defines the kind of value a variable can store
Variable	named storage used to hold a value
Operator	symbol that performs an operation
if / else	choose execution path based on condition
switch	choose among discrete case values
for	repeat using initialization, condition and update
while	repeat while a condition remains true
do-while	execute once, then check condition
break	exit the nearest loop/switch
continue	skip to the next loop iteration

Recommended GitHub folder:
JAVA-GIT / 02-Java-Basics-DataTypes-Variables-
Operators-Decision-Loops /